.NET ADVANCED INTERVIEW QUESTIONS 2026

Microservices, EF Core, Performance & Cloud

.NET Advanced Interview Questions (2026) – Microservices, EF Core, Performance & Scaling

Senior-Level Questions Asked in MNC Interviews

Need Deeper Guidance?
If you want real 1-on-1 help:
Complete Interview Q&A Resources, Live mock interviews,Career mentorship,Resume reviews,LinkedIn profile optimization Connect with me:

https://topmate.io/sandeeppal

Part 1 — Microservices Architecture

These are **real advanced concepts commonly asked in MNC interviews** such as Microsoft partners, product companies, fintech, and large enterprise systems.

---

1. What is Microservices Architecture and why is it used?

Answer

Microservices architecture is an architectural style where an application is divided into **small independent services**, each responsible for a **specific business capability**.

Each service:

- Runs independently
- Has its own database
- Can be deployed independently
- Communicates via APIs or messaging systems

Monolithic Architecture

https://www.linkedin.com/in/sandeeppal/

In a monolithic system everything is deployed as **one application**.

Example:

```
EcommerceApp
├── User Module
├── Order Module
├── Payment Module
└── Inventory Module
```

Problem with monoliths:

- Scaling entire system even if one module needs scaling
- Difficult deployments
- Large tightly coupled codebase

Microservices Version

User Service
Order Service
Payment Service
Inventory Service

Each service runs independently.

Real-world example

Netflix architecture:

Instead of one huge application, Netflix has **hundreds of microservices** like:

- Recommendation service
- Streaming service
- User service
- Billing service

Each service scales independently.

---

https://www.linkedin.com/in/sandeeppal/

2. When should you NOT use Microservices?

Interviewers often test this to see if you understand **architectural trade-offs**.

Microservices introduce **distributed system complexity**.

You should avoid microservices when:

- Small team (less than 5 developers)
- Early stage startup
- Simple CRUD application
- Low traffic system

Example

Internal HR portal:

Employee
Leave
Payroll

This can easily be a **modular monolith** instead of microservices.

Good Use Cases

Microservices work well when:

- System is very large
- Multiple teams work independently
- High scalability required
- Continuous deployments required

Example:

https://www.linkedin.com/in/sandeeppal/

Amazon
Uber
Netflix

---

3. What are the key components of a .NET Microservices architecture?

A production microservices architecture usually includes:

1 API Gateway

Acts as **single entry point**.

Responsibilities:

- Routing
- Authentication
- Rate limiting
- Aggregation

Examples:

- YARP (.NET)
- Ocelot
- Azure API Management

---

2 Service Discovery

When services scale dynamically, we cannot hardcode URLs.

Example:

Order Service → Payment Service

Service discovery tools:

https://www.linkedin.com/in/sandeeppal/

- Kubernetes DNS
- Consul
- Eureka

---

3 Message Broker

Used for **asynchronous communication**.

Examples:

- RabbitMQ
- Kafka
- Azure Service Bus

---

4 Distributed Logging

Since logs are spread across services.

Tools:

- ELK stack
- Seq
- Azure Monitor

---

5 Containerization

Microservices run inside containers.

Tools:

- Docker
- Kubernetes

https://www.linkedin.com/in/sandeeppal/

- AKS

---

4. What is an API Gateway and why is it important?

An **API Gateway** is the **entry point for all client requests**.

Instead of:

Client → Order Service
Client → Payment Service
Client → Product Service

We use:

Client → API Gateway → Microservices

Responsibilities

Routing

Example:

/orders → Order Service
/products → Product Service

Authentication

JWT validation can happen at gateway.

Rate Limiting

Prevent API abuse.

Example:

100 requests per minute per user

Aggregation

https://www.linkedin.com/in/sandeeppal/

Combines multiple responses.

Example:

Product page needs:

- product data
- reviews
- stock availability

Gateway aggregates these responses.

Example using Ocelot

```
{
"Routes": [
 {
  "DownstreamPathTemplate": "/api/orders",
  "UpstreamPathTemplate": "/orders",
  "DownstreamHostAndPorts": [
   {
    "Host": "orderservice",
    "Port": 80
   }
  ]
 }
]
}
```

---

5. What is the Saga Pattern?

Saga pattern is used to manage **distributed transactions in microservices**.

Traditional transactions cannot span across multiple databases.

Example scenario

Placing an order requires:

Order Service

https://www.linkedin.com/in/sandeeppal/

Payment Service
Inventory Service

Steps:

1 Order created
2 Payment processed
3 Inventory reserved

Failure Scenario

Payment succeeds but inventory fails.

We must **rollback previous actions**.

Saga Compensation

Reserve Inventory ❌
Refund Payment
Cancel Order

Two types

Choreography

Services react to events.

OrderCreated → Payment Service
PaymentCompleted → Inventory Service

Orchestration

Central service manages workflow.

Saga Orchestrator
↓
Order → Payment → Inventory

---

6. What is CQRS?

CQRS means **Command Query Responsibility Segregation**.

https://www.linkedin.com/in/sandeeppal/

We separate **read operations** and **write operations**.

Traditional model:

Single database
CRUD operations

CQRS model:

Command Model → Write Database
Query Model → Read Database

Benefits

- Faster reads
- Independent scaling
- Optimized models

Example

Write model:

Create Order
Cancel Order
Update Payment

Read model:

Order History
Order Dashboard
Analytics

---

7. What is Idempotency in APIs?

Idempotency means **multiple identical requests produce the same result**.

Problem

Network retry causes duplicate request.

https://www.linkedin.com/in/sandeeppal/

Example:

POST /payment

If request repeats:

User charged twice

Solution

Use **Idempotency Key**.

Idempotency-Key: 123456

Server stores processed request.

If same key repeats:

Return previous response.

Example (.NET)

```
if(cache.Contains(key))
{
return cache.Get(key);
}

var result = ProcessPayment();

cache.Set(key, result);

return result;
```

---

8. What is Synchronous vs Asynchronous communication?

Synchronous

Service calls another service using HTTP.

Example:

Order Service → Payment Service

https://www.linkedin.com/in/sandeeppal/

Pros:

- Immediate response
- Simple design

Cons:

- Tight coupling
- Failure propagation

Asynchronous

Services communicate through **events or message queues**.

Example:

OrderCreated Event → Queue → Payment Service

Tools:

- RabbitMQ
- Kafka
- Azure Service Bus

Real Example

Uber ride flow:

Ride Service
Driver Service
Payment Service
Notification Service

Events flow asynchronously.

---

https://www.linkedin.com/in/sandeeppal/

9. What is Circuit Breaker Pattern?

Circuit breaker prevents cascading failures.

Example:

Order Service → Payment Service

If payment service fails repeatedly:

Without circuit breaker:

System keeps retrying and overloads.

Circuit breaker states

Closed → normal traffic
Open → stop calls
Half-open → test recovery

.NET Example using Polly

```
var policy = Policy
.Handle<HttpRequestException>()
.CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));
```

---

10. What is Event Driven Architecture?

In event-driven architecture, services communicate via **events instead of direct API calls**.

Example:

OrderCreated Event

Consumers:

Payment Service
Email Service
Inventory Service

Benefits

https://www.linkedin.com/in/sandeeppal/

- Loose coupling
- Scalability
- Better fault tolerance

Technologies

- Kafka
- RabbitMQ
- Azure Event Hub
- Azure Service Bus

Part 2 — EF Core Advanced (Q11–20)

These questions are **commonly asked in MNC interviews for .NET developers** when the interviewer wants to evaluate your understanding of **ORM internals, performance optimization, database design, and production-scale applications**.

---

11. What is Entity Framework Core and how does it work internally?

Answer

Entity Framework Core (EF Core) is a lightweight, extensible, open-source **Object Relational Mapper (ORM)** for .NET.

It allows developers to interact with a database using **C# objects instead of writing raw SQL queries**.

Basic Workflow

C# Entity Classes

https://www.linkedin.com/in/sandeeppal/

↓
DbContext
↓
LINQ Queries
↓
EF Core Query Translator
↓
SQL Query
↓
Database

Example

Entity Model:

```
public class Product
{
   public int Id { get; set; }
   public string Name { get; set; }
   public decimal Price { get; set; }
}
```

DbContext:

```
public class AppDbContext : DbContext
{
   public DbSet<Product> Products { get; set; }
}
```

Query:

```
var products = context.Products.Where(p => p.Price > 100).ToList();
```

Generated SQL:

```
SELECT * FROM Products WHERE Price > 100
```

Internal Components

EF Core internally contains:

- **Change Tracker**
- **LINQ Provider**
- **Query Translator**
- **Database Provider (SQL Server, PostgreSQL, MySQL)**

These components translate LINQ queries into **optimized SQL statements**.

https://www.linkedin.com/in/sandeeppal/

Real-world Example

In a large **e-commerce platform**, EF Core manages entities like:

- Orders
- Customers
- Payments
- Products

Developers work with **objects**, while EF Core generates SQL queries automatically.

---

12. What is Change Tracking in EF Core?

Answer

Change Tracking is the mechanism EF Core uses to detect modifications made to entities so that it knows **what SQL operations to execute during SaveChanges().**

Example

```
var product = context.Products.First();

product.Price = 500;

context.SaveChanges();
```

EF Core tracks:

Old Price = 300
New Price = 500

Generated SQL:

```
UPDATE Products SET Price = 500 WHERE Id = 1
```

Entity States

https://www.linkedin.com/in/sandeeppal/

EF Core tracks entities using states:

State	Meaning
Added	Insert new record
Modified	Update record
Deleted	Delete record
Unchanged	No changes
Detached	Not tracked

Example

```
var entityEntry = context.Entry(product);
Console.WriteLine(entityEntry.State);
```

Real Production Scenario

In an **Order Processing System**, EF Core tracks:

Order Status Changed
Payment Status Updated
Inventory Updated

Then executes multiple SQL operations inside **one transaction**.

---

13. What is Tracking vs No-Tracking Query?

Answer

By default EF Core **tracks entities** in the Change Tracker.

But for **read-only operations**, tracking adds unnecessary overhead.

Tracking Query

https://www.linkedin.com/in/sandeeppal/

```
var products = context.Products.ToList();
```

EF Core stores entities in memory.

Benefits:

- Allows updates
- Tracks modifications

But uses **more memory and CPU**.

No Tracking Query

```
var products = context.Products
   .AsNoTracking()
   .ToList();
```

Benefits:

- Faster queries
- Less memory usage
- Better for read-heavy systems

Real-world Example

For **product catalog pages**:

Millions of read queries
No updates required

Using **AsNoTracking()** significantly improves performance.

---

14. What is the N+1 Query Problem in EF Core?

Answer

The **N+1 query problem** occurs when EF Core executes **one query for the parent entity and additional queries for each related entity.**

---

https://www.linkedin.com/in/sandeeppal/

Example

Entities:

Order
OrderItems

Code:

```
var orders = context.Orders.ToList();

foreach(var order in orders)
{
  var items = order.OrderItems;
}
```

SQL Generated

```
SELECT * FROM Orders
```

Then for each order:

```
SELECT * FROM OrderItems WHERE OrderId = 1
SELECT * FROM OrderItems WHERE OrderId = 2
SELECT * FROM OrderItems WHERE OrderId = 3
```

If there are **100 orders** → **101 queries**.

Solution

Use **Include()**.

```
var orders = context.Orders
  .Include(o => o.OrderItems)
  .ToList();
```

Generated SQL:

```
SELECT * FROM Orders
LEFT JOIN OrderItems
```

Real-world Example

In an **Order Dashboard**, failing to fix N+1 can increase response time from **100ms** → **several seconds**.

https://www.linkedin.com/in/sandeeppal/

---

15. What is Lazy Loading vs Eager Loading vs Explicit Loading?

Answer

EF Core provides **three ways to load related data**.

Lazy Loading

Data loads **automatically when accessed**.

Example:

```
var order = context.Orders.First();

var items = order.OrderItems;
```

EF Core executes a **second query automatically**.

Pros

Simple code.

Cons

Can cause **N+1 query problem**.

Eager Loading

Loads related data **immediately using Include()**.

```
var orders = context.Orders
  .Include(o => o.OrderItems)
  .ToList();
```

Best when related data is always required.

https://www.linkedin.com/in/sandeeppal/

Explicit Loading

Load related entities manually.

```
context.Entry(order)
    .Collection(o => o.OrderItems)
    .Load();
```

Used when loading conditionally.

---

16. What are Compiled Queries in EF Core?

Answer

EF Core compiles LINQ queries internally before executing SQL.

However, compiling queries repeatedly can impact performance in high-throughput systems.

Compiled queries cache the query plan.

Example

Normal Query:

```
var product = context.Products
  .FirstOrDefault(p => p.Id == id);
```

EF Core compiles query each time.

Compiled Query

```
static readonly Func<AppDbContext, int, Product> getProduct =
  EF.CompileQuery(
    (AppDbContext ctx, int id) =>
    ctx.Products.FirstOrDefault(p => p.Id == id));
```

Usage:

https://www.linkedin.com/in/sandeeppal/

```
var product = getProduct(context, id);
```

Benefits

- Faster execution
- Reduced CPU overhead
- Good for **high frequency queries**

Real Example

In a **high traffic API (1000+ requests/sec)** compiled queries improve performance significantly.

---

17. How does EF Core handle Transactions?

Answer

EF Core automatically wraps **SaveChanges() inside a transaction**.

Example

```
context.Orders.Add(order);
context.Payments.Add(payment);

context.SaveChanges();
```

Both operations succeed or fail together.

Manual Transaction

```
using var transaction = context.Database.BeginTransaction();

try
{
  context.Orders.Add(order);
  context.SaveChanges();
```

https://www.linkedin.com/in/sandeeppal/

```
    context.Payments.Add(payment);
    context.SaveChanges();

    transaction.Commit();
}
catch
{
    transaction.Rollback();
}
```

Real-world Example

Order creation requires:

Insert Order
Insert Payment
Update Inventory

All must succeed together.

---

18. What are Indexes and how do they improve EF Core performance?

Answer

An **index** is a database structure that improves query performance.

Without indexes, database performs **full table scan**.

Example

Query:

```
context.Users.Where(u => u.Email == email)
```

Without index:

Scan millions of rows

With index:

Direct lookup

https://www.linkedin.com/in/sandeeppal/

Creating Index

```
modelBuilder.Entity<User>()
  .HasIndex(u => u.Email);
```

Real-world Example

Large systems like **banking apps** require indexes for:

- Email lookup
- Order history
- Transaction search

Without indexes queries become extremely slow.

---

19. What are Bulk Operations and why are they important?

Answer

EF Core processes records **one by one**.

Example:

```
foreach(var product in products)
{
  context.Products.Add(product);
}

context.SaveChanges();
```

For large datasets this becomes slow.

Bulk Operations

Use specialized libraries:

https://www.linkedin.com/in/sandeeppal/

- EFCore.BulkExtensions
- Dapper
- Raw SQL

Example

```
context.BulkInsert(products);
```

Real-world Example

Importing **100,000 product records**.

Normal EF:

10+ minutes

Bulk insert:

Few seconds

---

20. What are some common EF Core performance optimization techniques?

Answer

Senior developers are expected to know these.

---

1 Use AsNoTracking for read queries

```
context.Products.AsNoTracking()
```

---

2 Use Projection instead of full entity

Bad:

```
context.Products.ToList()
```

https://www.linkedin.com/in/sandeeppal/

Better:

```
context.Products
.Select(p => new {p.Id, p.Name})
```

---

3 Add proper indexes

For frequently queried columns.

---

4 Avoid N+1 queries

Use **Include()**.

---

5 Use compiled queries

For high frequency queries.

---

6 Use pagination

Bad:

```
context.Products.ToList()
```

Better:

```
context.Products
.Skip(page * size)
.Take(size)
```

---

7 Use connection pooling

Handled automatically by ADO.NET.

Real-world Example

A production API serving **1 million daily users** improved response time from:

800 ms → 120 ms

after applying EF Core optimizations.

https://www.linkedin.com/in/sandeeppal/

21. What is asynchronous programming in .NET and why is it important for scalability?

Answer

Asynchronous programming allows a thread to **start an operation and continue executing other work without waiting for the operation to complete**.

In .NET this is mainly implemented using:

- `async`
- `await`
- `Task`

Problem with synchronous code

Example:

```
var data = GetDataFromDatabase();
var result = CallExternalAPI();
```

Each call blocks the thread.

If the API call takes **2 seconds**, the thread remains idle.

In high traffic systems this causes **thread starvation**.

Asynchronous version

```
var data = await GetDataFromDatabaseAsync();
var result = await CallExternalAPIAsync();
```

Here the thread returns to the **thread pool while waiting**.

https://www.linkedin.com/in/sandeeppal/

Real-world example

Imagine an API receiving **10,000 concurrent requests**.

If synchronous calls are used:

10,000 threads required

With async:

Few hundred threads handle the same workload

This dramatically improves scalability.

---

22. What is the Thread Pool in .NET?

Answer

The **Thread Pool** is a collection of worker threads managed by the .NET runtime.

Instead of creating new threads for every request, the runtime **reuses existing threads**.

Why thread pools exist

Creating threads is expensive:

- Memory allocation
- OS scheduling
- Context switching

Thread pool solves this by reusing threads.

Example

When an ASP.NET Core API receives a request:

Request → ThreadPool Worker Thread → Controller

https://www.linkedin.com/in/sandeeppal/

After completing the request, the thread returns to the pool.

Example Code

```
ThreadPool.QueueUserWorkItem(state =>
{
  Console.WriteLine("Running background task");
});
```

Real production example

A high traffic API may process:

50,000 requests per minute

Without thread pooling the server would **constantly create and destroy threads**, causing severe performance issues.

---

23. What is Garbage Collection (GC) in .NET?

Answer

Garbage Collection automatically **releases memory occupied by unused objects**.

In languages like C++ developers must manually free memory.

In .NET, the runtime handles this automatically.

How GC works

Objects are allocated in **managed heap**.

GC periodically checks:

Which objects are still referenced
Which objects are unused

Unused objects are removed.

https://www.linkedin.com/in/sandeeppal/

GC Generations

.NET uses **generational garbage collection**.

Generation	Description
Gen 0	Short-lived objects
Gen 1	Medium lifetime
Gen 2	Long-lived objects

Example

Short-lived objects:

```
var temp = new List<int>();
```

Usually collected quickly in **Gen 0**.

Large long-lived objects move to **Gen 2**.

Real-world example

In a web API processing thousands of requests:

Many objects are created per request.

Generational GC allows fast cleanup of temporary objects.

---

24. What is Memory Leak in .NET and how can it happen?

Answer

A memory leak occurs when objects **remain referenced even though they are no longer needed**, preventing GC from freeing memory.

---

https://www.linkedin.com/in/sandeeppal/

Common causes

Static references

```
public static List<Order> orders = new();
```

If orders are continuously added, memory keeps growing.

Event handlers not removed

```
publisher.Event += subscriber.Handler;
```

If the subscriber is never unsubscribed:

subscriber object never gets garbage collected

Caching without limits

```
MemoryCache.Set(key, value);
```

Without eviction policies memory grows indefinitely.

Real-world example

A background service storing large objects in static collections can eventually cause:

OutOfMemoryException

---

25. What is Response Caching in ASP.NET Core?

Answer

Response caching stores **HTTP responses so repeated requests can be served faster**.

Example

API endpoint:

GET /products

Instead of querying database every time, response is cached.

https://www.linkedin.com/in/sandeeppal/

Implementation

```
[ResponseCache(Duration = 60)]
public IActionResult GetProducts()
{
  return Ok(products);
}
```

This caches response for **60 seconds**.

Benefits

- Reduced database load
- Faster response times
- Improved scalability

Real-world example

Product catalog APIs often cache results for **30–60 seconds**.

This can reduce database traffic by **80% or more**.

---

26. What is Distributed Caching and why is it needed?

Answer

In large distributed systems, application instances run on **multiple servers**.

Local memory cache cannot be shared between instances.

Distributed cache solves this.

Example Architecture

https://www.linkedin.com/in/sandeeppal/

Client
↓
Load Balancer
↓
App Server 1
App Server 2
App Server 3

If each server caches data locally, cache becomes inconsistent.

Solution

Use centralized cache like:

- Redis
- Memcached

.NET Example using Redis

```
services.AddStackExchangeRedisCache(options =>
{
  options.Configuration = "localhost:6379";
});
```

Real-world example

Large e-commerce platforms store in Redis:

- session data
- product catalog
- user profiles

---

27. What is Output Caching in ASP.NET Core?

https://www.linkedin.com/in/sandeeppal/

Answer

Output caching stores the **entire HTTP response of an endpoint**.

Unlike response caching, it is **server controlled and more powerful**.

Example

```
[OutputCache(Duration = 60)]
public IActionResult GetProducts()
{
  return Ok(products);
}
```

Benefits

- Avoids database queries
- Avoids controller execution
- Improves response latency

Real example

For frequently accessed endpoints like:

GET /top-products
GET /featured-products

Output caching can significantly reduce server load.

---

28. What are the best practices to improve API performance in ASP.NET Core?

Answer

https://www.linkedin.com/in/sandeeppal/

Senior developers are expected to know these.

Use async everywhere

Avoid blocking threads.

Use pagination

Bad:

```
context.Products.ToList();
```

Better:

```
context.Products
.Skip(page * size)
.Take(size);
```

Use caching

- Redis
- MemoryCache

Optimize database queries

Avoid:

```
SELECT *
```

Use projections.

Enable compression

```
services.AddResponseCompression();
```

Use connection pooling

Handled automatically by ADO.NET.

https://www.linkedin.com/in/sandeeppal/

Real-world example

An API handling **1M daily requests** reduced latency from:

600 ms → 120 ms

after applying these optimizations.

---

29. What is Load Balancing?

Answer

Load balancing distributes incoming traffic across **multiple servers**.

Example

Without load balancer:

Client → Server

Server becomes overloaded.

With load balancer

Client
↓
Load Balancer
↓ ↓ ↓
Server1 Server2 Server3

Traffic is distributed.

Types of load balancing

Round Robin
Least Connections
IP Hash

https://www.linkedin.com/in/sandeeppal/

Real-world example

Large systems like **Amazon or Netflix** run thousands of servers behind load balancers.

---

30. What is Horizontal vs Vertical Scaling?

Answer

Scaling is the process of increasing system capacity.

Vertical Scaling

Increase power of a single machine.

Example:

8 GB RAM → 64 GB RAM

Limitations:

- Hardware limit
- Expensive

Horizontal Scaling

Add more servers.

Example:

1 Server → 10 Servers

Benefits:

- Better fault tolerance

https://www.linkedin.com/in/sandeeppal/

- Infinite scalability

Example Architecture

Client
↓
Load Balancer
↓
Multiple App Servers
↓
Database Cluster

Real-world example

Large systems scale horizontally using:

- Kubernetes
- Docker containers
- Cloud auto-scaling

31. What is a Distributed System?

Answer

A **distributed system** is a system where multiple independent computers work together and appear as a **single system to the user**.

Instead of one server handling everything, the workload is distributed across multiple machines.

Example Architecture

Client
↓
Load Balancer
↓
API Gateway
↓
Microservices
├── User Service

https://www.linkedin.com/in/sandeeppal/

```
├── Order Service
├── Payment Service
└── Notification Service
```

Each service runs on different servers.

Key Characteristics

Scalability
Fault tolerance
High availability
Independent deployment

Real-world Example

Large platforms like **Amazon or Uber** consist of **thousands of distributed services** communicating through APIs and message queues.

---

32. What is the CAP Theorem?

Answer

CAP theorem states that a distributed system can guarantee **only two out of the following three properties** at the same time.

Property	Description
Consistency	All nodes see the same data
Availability	Every request gets a response
Partition Tolerance	System continues working despite network failures

---

CAP Trade-off

A system must choose:

CP → Consistency + Partition tolerance
AP → Availability + Partition tolerance

---

https://www.linkedin.com/in/sandeeppal/

Example

Banking systems

Consistency > Availability

Balance must always be correct.

Social media platforms

Availability > Consistency

Seeing slightly outdated posts is acceptable.

---

33. What is Eventual Consistency?

Answer

Eventual consistency means that **data will become consistent across all nodes after some time**, even if temporary inconsistencies occur.

This model is commonly used in distributed systems.

Example

User updates profile name.

Update Request → Database A

Other replicas update later.

Database B
Database C

For a short time some nodes may still show old data.

Eventually all databases sync.

Real-world Example

https://www.linkedin.com/in/sandeeppal/

Systems like **Amazon DynamoDB and Cassandra** use eventual consistency to achieve massive scalability.

---

34. What is Message Queue and why is it used?

Answer

A **message queue** enables asynchronous communication between services.

Instead of calling another service directly, a service sends a message to a queue.

Example Flow

Order Service
↓
Message Queue
↓
Payment Service

Order service does not wait for payment service to complete.

Benefits

Loose coupling
Better scalability
Improved fault tolerance

Popular Message Brokers

RabbitMQ
Kafka
Azure Service Bus
AWS SQS

Real-world Example

https://www.linkedin.com/in/sandeeppal/

In an **e-commerce system**:

Order Created Event

Triggers:

- Payment processing
- Email notification
- Inventory update

All through message queues.

---

35. What is Event Sourcing?

Answer

Event sourcing is a pattern where **state changes are stored as a sequence of events instead of updating records directly**.

Traditional Model

Account Balance = 500

Update:

Balance = 600

Old data is lost.

Event Sourcing Model

AccountCreated
MoneyDeposited 500
MoneyDeposited 100
MoneyWithdrawn 50

Current state is calculated from events.

---

https://www.linkedin.com/in/sandeeppal/

Benefits

Complete audit history
Easy debugging
Time travel debugging

Real-world Example

Banking systems store events like:

Deposit
Withdrawal
Transfer

Instead of just updating the balance.

---

36. What is API Rate Limiting?

Answer

Rate limiting restricts how many requests a client can make within a given time.

This protects APIs from:

- abuse
- DDoS attacks
- accidental traffic spikes

Example

Limit:

100 requests per minute per user

If exceeded:

HTTP 429 Too Many Requests

---

https://www.linkedin.com/in/sandeeppal/

ASP.NET Core Example

```
builder.Services.AddRateLimiter(options =>
{
  options.AddFixedWindowLimiter("api", limiterOptions =>
  {
     limiterOptions.PermitLimit = 100;
     limiterOptions.Window = TimeSpan.FromMinutes(1);
  });
});
```

Real-world Example

Public APIs like **GitHub or Twitter APIs** enforce strict rate limits.

---

37. What is Distributed Tracing?

Answer

In microservices, a request often travels across multiple services.

Example request flow:

Client
↓
API Gateway
↓
Order Service
↓
Payment Service
↓
Notification Service

Debugging becomes difficult.

Distributed tracing helps track the request across services.

Popular Tools

OpenTelemetry
Jaeger
Zipkin
Azure Application Insights

https://www.linkedin.com/in/sandeeppal/

Real-world Example

A request ID travels through all services:

TraceId = 12345

Developers can trace the entire request lifecycle.

---

38. What is Service Mesh?

Answer

A **service mesh** manages communication between microservices.

Instead of handling networking logic inside services, it is handled by a **sidecar proxy**.

Architecture

Service A → Sidecar Proxy → Network → Sidecar Proxy → Service B

Responsibilities

Traffic routing
Load balancing
Security (mTLS)
Observability

Popular Service Mesh Tools

Istio
Linkerd
Consul

Real-world Example

Large Kubernetes clusters use **Istio** to control service communication and monitoring.

https://www.linkedin.com/in/sandeeppal/

---

39. What is Containerization and why is it important?

Answer

Containerization packages an application along with its dependencies into a **portable container**.

This ensures the application runs consistently across environments.

Example Problem

Without containers:

Works on developer machine
Fails on production server

Docker Container

Application
Runtime
Dependencies
OS libraries

All packaged together.

Benefits

Environment consistency
Easy deployment
Better resource utilization

Real-world Example

Most modern .NET microservices are deployed using **Docker containers**.

---

https://www.linkedin.com/in/sandeeppal/

40. What is Kubernetes and why is it used with .NET applications?

Answer

Kubernetes is a **container orchestration platform** used to manage containerized applications.

It automates:

- deployment
- scaling
- networking
- failover

Kubernetes Architecture

```
Users
↓
Load Balancer
↓
Kubernetes Cluster
  ├── Pod (.NET Service)
  ├── Pod (.NET Service)
  ├── Pod (.NET Service)
```

Key Features

Auto-scaling
Self-healing containers
Rolling deployments
Service discovery

Example

If a container crashes:

Kubernetes automatically creates a new one

https://www.linkedin.com/in/sandeeppal/

Real-world Example

Large enterprise systems run **hundreds of microservices on Kubernetes clusters** for high availability and scalability.

41. What is Dependency Injection (DI) in .NET and why is it important?

Answer

Dependency Injection (DI) is a design pattern used to achieve **loose coupling between classes**.

Instead of creating dependencies inside a class, dependencies are **provided externally**.

Problem without DI

```
public class OrderService
{
  private PaymentService payment = new PaymentService();
}
```

Problems:

- Tight coupling
- Hard to test
- Hard to replace implementation

Using Dependency Injection

```
public class OrderService
{
  private readonly IPaymentService _payment;

  public OrderService(IPaymentService payment)
```

https://www.linkedin.com/in/sandeeppal/

```
    {
        _payment = payment;
    }
}
```

Dependency is injected from outside.

.NET Built-in DI Example

```
builder.Services.AddScoped<IPaymentService, PaymentService>();
```

Real-world Example

In large enterprise systems:

OrderService → PaymentService
OrderService → EmailService
OrderService → InventoryService

DI allows **easily swapping implementations**.

Example:

PaymentService → StripePaymentService
PaymentService → PayPalPaymentService

---

42. What are Service Lifetimes in ASP.NET Core?

Answer

ASP.NET Core DI container supports **three service lifetimes**.

Lifetime	Description
Transient	New instance every time
Scoped	One instance per HTTP request
Singleton	One instance for entire application

---

https://www.linkedin.com/in/sandeeppal/

Transient

services.AddTransient<IService, Service>();

Created every time it is requested.

Used for:

- lightweight services
- stateless services

Scoped

services.AddScoped<IService, Service>();

One instance per HTTP request.

Most commonly used for:

DbContext
Repository
Business services

Singleton

services.AddSingleton<IService, Service>();

Single instance for entire app lifetime.

Used for:

- caching services
- configuration services

Common Interview Trap

DbContext should never be Singleton.

Because DbContext is **not thread safe**.

---

https://www.linkedin.com/in/sandeeppal/

43. What is Middleware in ASP.NET Core?

Answer

Middleware is a component that **handles HTTP requests and responses in the ASP.NET Core pipeline**.

Each middleware can:

- process request
- call next middleware
- modify response

Request Pipeline

Client
↓
Middleware 1
↓
Middleware 2
↓
Middleware 3
↓
Controller

Example Middleware

```
app.Use(async (context, next) =>
{
  Console.WriteLine("Request started");

  await next();

  Console.WriteLine("Response finished");
});
```

Built-in Middleware

https://www.linkedin.com/in/sandeeppal/

Authentication
Authorization
Routing
Exception handling
CORS

Real-world Example

Typical ASP.NET Core pipeline:

Exception Middleware
Authentication
Authorization
Routing
Controller

---

44. What is Authentication vs Authorization?

Answer

These two concepts are often confused.

Authentication

Authentication verifies **who the user is**.

Example:

Login with username and password
Login with Google
Login with JWT token

Authorization

Authorization determines **what the user is allowed to do**.

Example:

Admin → create products

https://www.linkedin.com/in/sandeeppal/

User → view products

ASP.NET Core Example

Authentication:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme);
```

Authorization:

```
[Authorize(Roles = "Admin")]
```

Real-world Example

In an e-commerce platform:

Customer → place orders
Admin → manage products

Authorization ensures correct access control.

---

45. What is JWT Authentication?

Answer

JWT (JSON Web Token) is a **token-based authentication mechanism** widely used in modern APIs.

Instead of storing sessions on the server, authentication information is stored in a **signed token**.

JWT Structure

A JWT consists of three parts:

Header.Payload.Signature

Example:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

---

https://www.linkedin.com/in/sandeeppal/

Authentication Flow

User Login
↓
Server validates credentials
↓
Server generates JWT token
↓
Client stores token
↓
Client sends token in API requests

Header example:

Authorization: Bearer <token>

Benefits

Stateless authentication
Scales well in microservices
No server session storage

---

46. What is Clean Architecture?

Answer

Clean Architecture separates application code into **layers to maintain independence and maintainability**.

Typical Layers

Presentation Layer
Application Layer
Domain Layer
Infrastructure Layer

Layer Responsibilities

Presentation → Controllers / APIs

Application → Business logic

https://www.linkedin.com/in/sandeeppal/

Domain → Entities and business rules

Infrastructure → Database, external APIs

Dependency Rule

Dependencies must point **inward**.

Infrastructure → Application → Domain

Domain must remain independent.

Real-world Example

In a large banking system:

Domain → Account, Transaction
Application → TransferMoneyService
Infrastructure → SQL Server repository

Domain logic remains independent of database.

---

47. What is the Repository Pattern and should it always be used?

Answer

Repository pattern abstracts database operations.

Instead of accessing DbContext directly:

```
context.Products.Add(product);
```

Use repository:

```
productRepository.Add(product);
```

Benefits

https://www.linkedin.com/in/sandeeppal/

Encapsulates data access logic
Improves testability
Decouples business logic from ORM

Debate in modern .NET

Many developers argue that:

EF Core already acts as a repository + unit of work

So creating additional repositories sometimes becomes unnecessary.

Best Practice

Use repository when:

- multiple data sources exist
- complex data access logic exists

---

48. What is the difference between EF Core and Dapper?

Answer

Both are used for database access but they serve different purposes.

Feature	EF Core	Dapper
Type	Full ORM	Micro ORM
Performance	Slower	Faster
Features	Change tracking, migrations	Minimal
Query Control	Less control	Full SQL control

EF Core Example

https://www.linkedin.com/in/sandeeppal/

```
var products = context.Products.ToList();
```

Dapper Example

```
var products = connection.Query<Product>("SELECT * FROM Products");
```

Real-world Strategy

Many systems use **both**.

Example:

EF Core → CRUD operations
Dapper → high-performance queries

---

49. What is the difference between IEnumerable, IQueryable, and List?

Answer

These are frequently asked LINQ interview questions.

IEnumerable

Works with **in-memory collections**.

Example:

```
IEnumerable<Product> products = context.Products.ToList();
```

Filtering happens in memory.

IQueryable

Used for **database queries**.

Query translated to SQL.

```
IQueryable<Product> products = context.Products.Where(p => p.Price > 100);
```

https://www.linkedin.com/in/sandeeppal/

SQL generated:

```
SELECT * FROM Products WHERE Price > 100
```

List

Concrete collection type.

```
List<Product> products = context.Products.ToList();
```

All data is already loaded.

Key Difference

IQueryable → executes in database
IEnumerable → executes in memory

---

50. What are common mistakes developers make in large .NET systems?

Answer

Interviewers often ask this to evaluate **real-world experience**.

---

1 Overusing microservices

Not every system needs microservices.

Sometimes **modular monolith is better**.

---

2 Poor database design

Lack of indexes causes slow queries.

---

3 Blocking async code

Example:

https://www.linkedin.com/in/sandeeppal/

Task.Result
Task.Wait()

This causes **thread starvation**.

---

4 Not using caching

For high traffic APIs caching is essential.

---

5 Ignoring observability

Large systems require:

Centralized logging
Distributed tracing
Metrics monitoring

Real-world Example

A poorly optimized API may experience:

High CPU usage
Slow response times
Frequent crashes

Proper architecture and performance practices prevent these issues.

Need Deeper Guidance?
If you want real 1-on-1 help:
Complete Interview Q&A Resources, Live mock interviews,Career mentorship,Resume reviews,LinkedIn profile optimization Connect with me:

https://topmate.io/sandeeppal

https://www.linkedin.com/in/sandeeppal/